

Proyecto de Calidad de Software

Fase 3

Sistema POS de entregas para PYMEs – Delivo

Aaron Santana Valdeloamr
Diego Cerdas Delgado

2 de julio de 2026

Repositorio: `tributocr-mobile` (aplicación móvil “Delivo”)

Índice

1. Introducción	3
2. Marco metodológico	3
2.1. Características del proyecto	3
2.2. Arquitectura	3
2.3. Proceso de desarrollo en esta iteración	3
3. Calidad general del producto	4
3.1. Repositorio y versionamiento	4
3.2. Casos de uso seleccionados	4
3.2.1. PBI-003: Registro y Gestión de Órdenes de Pedido	4
3.2.2. PBI-004: Inteligencia Logística y Optimización de Rutas de Entrega	4
3.3. Casos de prueba	5
3.4. Pruebas automatizadas y cobertura	5
3.4.1. Hallazgo: defecto encontrado y corregido	5
3.5. Métricas de GQM ejecutadas	6
4. Análisis estático	6
4.1. ESLint	6
4.2. Sentry	6
4.3. Resultados	7
4.4. Comparación	7
5. Conclusiones	7

1 Introducción

Continuando el plan de calidad definido en las Fases 1 y 2 para **Delivo**, en esta iteración se seleccionaron dos casos de uso del *product backlog* para programarlos y verificarlos de forma completa:

1. **PBI-003 – Registro y Gestión de Órdenes de Pedido.**
2. **PBI-004 – Inteligencia Logística y Optimización de Rutas de Entrega.**

Ambos ya contaban con una implementación funcional heredada del desarrollo incremental del producto. El trabajo de esta fase se concentró en dotarlos de pruebas automatizadas reales (no existían antes en el repositorio) y en ejecutar dos herramientas de análisis de código sobre el proyecto completo. Las métricas de GQM que se reportan con evidencia real son las del **Objetivo 3 (mantenibilidad)**, por ser el que corresponde directamente a los dos casos de uso trabajados; los objetivos 1, 2 y 4 dependen de módulos que no se tocaron en esta iteración y quedan pendientes de medir cuando se desarrollen.

2 Marco metodológico

2.1 Características del proyecto

Atributo	Valor
Nombre del producto	Delivo (repositorio <code>tributocr-mobile</code>)
Framework	Expo SDK ~56, React Native 0.85.3
Lenguaje	TypeScript ~5.9.2, modo <code>strict</code>
Enrutamiento	<code>expo-router</code> (basado en archivos)
Mapas / ruteo	<code>@rnmapbox/maps</code> + <code>@mapbox/polyline</code>
Persistencia	Sin base de datos embebida; backend REST remoto consumido vía <code>axios</code> (el cliente móvil es un <i>thin client</i>)
Monitoreo	<code>@sentry/react-native</code> y <code>expo-observe</code>
Tamaño del código	153 archivos <code>.ts/.tsx</code> , 17712 líneas de código
Modelos de dominio	9 interfaces en <code>types/*.d.ts</code> : <code>Order</code> , <code>Route</code> , <code>Client</code> , <code>Company</code> , <code>Product</code> , <code>PaymentMethod</code> , <code>User</code> , <code>FinancialSummary</code> , <code>Location</code>

2.2 Arquitectura

El proyecto usa una arquitectura en capas organizada por dominio: pantallas y componentes de UI (`app/`, `components/`), estado con Context API y *hooks* de dominio (`context/`, `hooks/`) que encapsulan *fetch* + estado de carga/error, y una capa de servicios (`services/`) donde cada dominio hereda de una clase base común (`BaseApiService`) que envuelve los verbos HTTP. No es Redux ni Clean Architecture formal, pero separa responsabilidades de forma clara y hace que los servicios sean fácilmente sustituibles por *mocks* en pruebas, como se aprovechó en la Sección 3.4.

2.3 Proceso de desarrollo en esta iteración

De las diez actividades de V&V definidas en la Fase 2, las que se ejecutaron concretamente sobre los dos casos de uso de esta fase fueron:

- **Análisis estático y *code review*** (Desarrollador Principal): se ejecutó ESLint sobre las 153 archivos del proyecto – resultados en la Sección 4.

- **Pruebas unitarias del módulo *core*** (Desarrollador de Software): se configuró por primera vez la infraestructura de pruebas (Jest) y se escribieron 35 pruebas para los dos casos de uso – resultados en la Sección 3.4.

Al ser un equipo de dos personas, la revisión de pares se hizo directamente sobre el código y las pruebas producidas, cada una trazable a un criterio de aceptación por su identificador (CP-ORD-XX / CP-RUT-XX). Esta revisión tuvo un efecto concreto: escribir las pruebas de rutas encontró un defecto real en producción, documentado en la Sección 3.4.1.

3 Calidad general del producto

3.1 Repositorio y versionamiento

- **Ramas:** *main*, *development*, *production*, con flujo *feature branch* → *Pull Request* → *merge*.
- **Convención de commits:** Conventional Commits en la mayoría de los 126 commits de *development* (72 *feat*, 6 *fix*, 3 *chore*, 1 *style*, más los *merges* de PR).
- **Contribuciones:** 124 commits de Aaron Santana (desarrollo incremental del producto en fases previas) y 2 commits de Diego Cerdas correspondientes al trabajo de esta iteración (infraestructura de pruebas y suite de pruebas con la corrección del defecto encontrado).
- **coverage/** se agregó a *.gitignore* para no versionar artefactos generados por Jest.

3.2 Casos de uso seleccionados

3.2.1 PBI-003: Registro y Gestión de Órdenes de Pedido

Descripción:

Como Repartidor o Administrador, quiero registrar nuevas órdenes ingresando el cliente, producto, cantidad y dirección, para mantener el control digitalizado de los pedidos pendientes de entrega.

Implementación: pantalla `app/orders/create.tsx`, servicio `services/orders/Orders.service.ts` (`create`, `addItemToOrder`, `getOrderById`, `all`, entre otros), estado con `hooks/useOrders.ts` y modelo `types/Order.d.ts` (`OrderStatus`: `PENDING`/`PAID`/`CANCELLED`).

3.2.2 PBI-004: Inteligencia Logística y Optimización de Rutas de Entrega

Descripción:

Como Administrador, quiero crear, organizar y optimizar las rutas de entrega basándome en la ubicación geográfica de los clientes, para minimizar la distancia recorrida y eliminar la planificación empírica.

Implementación: pantallas en `app/routes/`, servicio `services/routes/Routes.service.ts` (`create`, `find`, `addPoint`, `startNavigation`, `completeDelivery`), estado con `context/RouteContext.tsx` (ordena las paradas y rastrea la entrega activa) y modelo `types/Route.d.ts`. La optimización de la secuencia de paradas y la polilínea se calculan en el backend; el cliente móvil solo decodifica y renderiza el resultado.

3.3 Casos de prueba

Se automatizaron 22 casos de prueba entre los dos casos de uso; los más representativos:

ID	Descripción	Entrada	Resultado esperado
CP-ORD-01	Crear una orden asociada a un cliente	<code>clientId: 'client-1'</code>	Orden creada con <code>status = PENDING</code>
CP-ORD-08	Buscar órdenes por cliente o número	texto <code>."esquina" / "202"</code>	Listado filtrado correctamente, sin distinguir mayúsculas
CP-RUT-01	Crear una ruta nueva	<code>{ name: 'Ruta Centro' }</code>	Ruta creada con <code>status = CREATED</code>
CP-RUT-09	Ordenar las paradas al obtener la ruta	<code>points</code> con índices desordenados	Puntos ordenados ascendentemente por <code>index</code>
CP-RUT-11	Completar la última parada pendiente	Ruta de una parada, entrega completada	<code>status</code> pasa a <code>COMPLETED</code>

3.4 Pruebas automatizadas y cobertura

El repositorio no tenía ninguna prueba automatizada antes de esta fase. Se configuró `jest-expo 56.0.5`, `@testing-library/react-native 13.3.3` y `jest 29.7.0`, con la cobertura acotada a los cinco módulos donde vive la lógica de negocio de los dos casos de uso (medir el 100 % de la aplicación, incluyendo pantallas de módulos no tocados en esta iteración, habría diluido la métrica sin aportar evidencia real).

Archivo	% Sentencias	% Funciones	% Líneas
<code>Orders.service.ts</code>	100 %	100 %	100 %
<code>Routes.service.ts</code>	100 %	100 %	100 %
<code>useOrders.ts</code>	96.29 %	100 %	100 %
<code>useRoutes.ts</code>	95.00 %	100 %	100 %
<code>RouteContext.tsx</code>	93.82 %	100 %	98.59 %
Total (392 LOC, 35 pruebas)	95.42 %	100 %	99.28 %

La cobertura por sentencias, funciones y líneas supera el 90 % exigido (la cobertura de ramas quedó en 70.45 %, menor, porque algunas combinaciones de manejo de error se priorizaron menos frente al tiempo disponible).

3.4.1 Hallazgo: defecto encontrado y corregido

Al escribir las pruebas de `RouteContext.tsx` se detectó que la ruta nunca se marcaba como `COMPLETED` cuando la primera parada pendiente estaba en el índice 0 (el caso más común, toda ruta recién creada). La causa: usar `||` para un valor que puede ser legítimamente 0, que en JavaScript es *falsy*.

Listing 1: `context/RouteContext.tsx`: antes y después de la corrección

```

1 // Antes: "findIndex(...) || null" convierte el índice 0 en null
2 setCurrentPointIndex(orderedPoints.findIndex(p => p.status === 'CREATED') || null);
3
4 // Despues: comparacion explicita en vez de coercion implicita
5 const pendingIndex = orderedPoints.findIndex(p => p.status === 'CREATED');
6 setCurrentPointIndex(pendingIndex === -1 ? null : pendingIndex);

```

Sin esta corrección, un repartidor que completaba la última parada de una ruta nunca veía la ruta transicionar a **COMPLETED**. Es la evidencia más concreta de que las pruebas automatizadas de esta iteración tuvieron valor real más allá de una cifra de cobertura.

3.5 Métricas de GQM ejecutadas

Del marco GQM de la Fase 2, el Objetivo 3 (mantenibilidad) es el medible con la evidencia de esta iteración:

Métrica M3.2 – Complejidad Ciclomática	Valor	Rango (Fase 2)
Promedio (20 funciones medidas con ESLint, regla <code>complexity</code>)	3.05	Óptimo (1–10)
Máxima	5	Óptimo

El objetivo se cumple para el código de esta iteración: ninguna función supera 5, muy por debajo del umbral crítico de 20. La métrica M3.1 (Índice de Mantenibilidad) no pudo calcularse: requiere el Volumen de Halstead, y las herramientas disponibles para eso (`complexity-report`, `escomplex`) no soportan `async/await` ni sintaxis moderna de TypeScript — se documenta como limitación real en vez de reportar un número sin respaldo. Los objetivos 1, 2 y 4 (facturación, visibilidad en exteriores, datos sensibles) dependen de módulos fuera del alcance de esta iteración y quedan pendientes de medir.

4 Análisis estático

El análisis estático examina el código fuente sin ejecutarlo, para detectar errores, malas prácticas o patrones riesgosos antes de llegar a producción; se complementa con el análisis dinámico, que observa el sistema en ejecución. Para esta entrega se usaron **ESLint** (estático) y **Sentry** (monitoreo dinámico de errores en producción), ya integrado y usado activamente en el producto real. Se incluye pese a no ser una herramienta estática porque permite comparar dos filosofías de calidad complementarias, como se muestra en la Sección 4.4.

4.1 ESLint

Creado por Nicholas C. Zakas en 2013, ESLint es el *linter* estándar del ecosistema JavaScript/TypeScript: construye un árbol de sintaxis (AST) y lo evalúa contra reglas configurables de estilo, buenas prácticas y errores probables. El proyecto usa `eslint 9.25.0` (*flat config*) con el preset `eslint-config-expo 10.0.0`, ejecutado con `npm run lint`.

4.2 Sentry

Originada en 2008 como herramienta interna de Disqus y luego liberada como proyecto de código abierto, Sentry captura excepciones y *crashes* de una aplicación en ejecución, con contexto de dispositivo/usuario y repetición visual de la sesión (*session replay*). El proyecto usa `@sentry/react-native ~7.11.0`, inicializado en `app/_layout.tsx`:

Listing 2: Inicialización de Sentry (`app/_layout.tsx`)

```

1 Sentry.init({
2   dsn: 'https://...@4511549122084864.ingest.us.sentry.io/...',
3   replaysSessionSampleRate: 0.1,
4   replaysOnErrorSampleRate: 1,
5   integrations: [Sentry.mobileReplayIntegration(), Sentry.feedbackIntegration()],
6 });

```

```
7 export default Sentry.wrap(ObserveRoot.wrap(RootLayout));
```

Además del *wrap* global, `components/feedback/ErrorBoundary.tsx` captura explícitamente los errores de renderizado de React vía `Sentry.captureException`.

4.3 Resultados

Regla de ESLint	Categoría	Ocurrencias
@typescript-eslint/no-unused-vars	Código muerto	35
react-hooks/exhaustive-deps	<i>Hooks</i>	17
@typescript-eslint/array-type	Estilo	2
import/no-named-as-default-member	Importaciones	2
Total	0 errores, 56 advertencias	(165 archivos, 37 con alguna advertencia)

Cero errores en todo el proyecto es una señal fuerte de calidad; las advertencias son deuda técnica localizable, sin bloquear el *build*.

Sobre Sentry: solo dos archivos de todo el proyecto lo invocan (`app/_layout.tsx` y `ErrorBoundary.tsx`). Ningún `catch` de `services/` o `hooks/` — incluyendo los de Órdenes y Rutas — reporta a Sentry; todos usan solo `console.error`. En producción, un fallo de red al crear una orden no genera ningún evento en Sentry, solo un *toast* para el usuario.

4.4 Comparación

	ESLint	Sentry
Tipo	Estático, sin ejecución	Dinámico, en producción
Cuándo detecta	En cada <i>commit</i> /PR	Solo si el código defectuoso se ejecuta y falla
Qué detecta	Código muerto, malas prácticas, complejidad	Excepciones y <i>crashes</i> reales, con contexto de sesión
Falsos negativos	No detectó el bug de la Sección 3.4.1 (sintaxis válida)	Ciego a errores que solo se registran con <code>console.error</code>
Costo	Gratuito, sin infraestructura	Cuota de eventos/ <i>replays</i> en el plan pagado

Ambas herramientas son factibles en proyectos grandes: ESLint escala sin costo adicional y es el estándar de facto del ecosistema; Sentry gana valor con el tráfico real, pero su utilidad depende de que el equipo instrumente el código de negocio, no solo el nivel superior de la aplicación.

5 Conclusiones

La calidad de los dos casos de uso trabajados se sustenta en evidencia verificable: pruebas automatizadas reales que antes no existían (35 casos, cobertura de sentencias/funciones/líneas sobre el 90 % exigido), análisis estático limpio (0 errores de ESLint, complejidad ciclomática baja) y un defecto funcional real detectado y corregido gracias a las pruebas.

Lecciones aprendidas:

- El análisis estático y las pruebas dinámicas no son intercambiables: el bug de `RouteContext.tsx` era sintácticamente válido y solo una prueba de comportamiento lo reveló.
- Adoptar pruebas en un proyecto que nunca las tuvo tiene un costo de arranque (versión correcta de `jest-expo`, resolución de dependencias nativas), pero la capa de servicios y *hooks*, al estar desacoplada de la UI, fue rápida de probar.
- Tener Sentry integrado no garantiza observabilidad si no se instrumenta en todas las capas; instrumentar los `catch` de la capa de servicios queda como mejora para la siguiente iteración.
- Medir solo lo que hay evidencia real para medir — en vez de reportar los cuatro objetivos de GQM cuando solo dos casos de uso se trabajaron — es en sí una práctica de calidad.